

CODE WALKTHROUGH · YOUR REPO

What the important code is doing

This is not a TensorFlow tutorial. It is a guided tour of **gan-fraud-detection**. Open the file named in teal, look at the snippet, then read the 'why' lines. The crash-guide PDF was the ideas. This one is the code.

How to use this

You do not need to memorise every line. You need to be able to point at `src/gan.py` and say: this builds G, this builds C, this is the gradient penalty, this is `-mean(C(fake))`, this is five critic steps per generator step. If you can do that, you can defend the repo.

0. Map of the repo — what to ignore

Read files in this order. Everything else is plumbing (paths, plots, JSON dumps).

File	Job	Skip?
<code>run.py</code>	Starts each stage in order: data → gan → detect.	Read once.
<code>src/config.py</code>	All knobs ($\lambda=10$, $n_critic=5$, 4000 steps).	Skim the numbers.
<code>src/data.py</code>	Download, split FIRST, scale on train only.	The leakage rule lives here.
<code>src/gan.py</code>	Generator, critic, GP, both losses.	This is the project.
<code>src/train_gan.py</code>	The 5:1 training loop on train fraud only.	Read with <code>gan.py</code> .
<code>src/generate.py</code>	After training: noise in, fake frauds out.	Short.
<code>src/train_detector.py</code>	Three XGBs, same test set.	The proof.
<code>src/evaluate.py</code>	AUPRC + val F1 threshold.	Short.
<code>src/infer.py</code>	Scaler + XGB. No generator.	What ships.
<code>src/plots.py</code>	Loss / PR / t-SNE figures.	Skip the internals.

1. run.py — the movie, not the plot

run.py

This file does not train anything itself. It sets the environment, then calls the right module. `python run.py all` is just these four ifs in a row.

```
os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1")
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")

if args.stage in ("all", "data"):
    from src.data import prepare
    prepare()

if args.stage in ("all", "gan"):
    from src.train_gan import train as train_gan
    from src.generate import generate
    train_gan()
    generate()

if args.stage in ("all", "detect"):
    from src.train_detector import train as train_detect
    train_detect()
```

CUDA_VISIBLE_DEVICES=-1 — Force CPU. This MLP does not need a GPU.

TF_CPP_MIN_LOG_LEVEL=2 — Hide TensorFlow's CUDA warnings so logs are readable.

`prepare()` — Split + scale. Must finish before the GAN starts.
`train_gan(); generate()` — Learn the fraud blob, then print extra rows.
`train_detect()` — Fit three XGBoosts and score the frozen test set.

Imports sit **inside** the ifs so python `run.py` data does not even load TensorFlow.

2. config.py — the knobs that match the papers

`src/config.py`

```
NOISE_DIM = 32          # size of z, the generator's random seed
G_HIDDEN = (128, 256)   # generator MLP widths
C_HIDDEN = (256, 128)   # critic MLP widths
BATCH_SIZE = 64
G_STEPS = 4000          # generator updates (not epochs)
N_CRITIC = 5            # critic updates per generator update
GP_LAMBDA = 10.0        # how hard we tax a bad critic slope
GAN_LR = 1e-4
GAN_BETA_1 = 0.0        # WGAN-GP paper: Adam with  $\beta_1 = 0$ 
GAN_BETA_2 = 0.9
SYNTH_MULTIPLIER = 8    # print 8x as many fakes as real train frauds
```

N_CRITIC = 5 — Keep the judge slightly smarter than the forger. 1:1 often collapses.

GP_LAMBDA = 10 — Standard WGAN-GP value. Do not tune this on day one.

GAN_BETA_1 = 0.0 — Momentum in Adam fights the adversarial game. The paper turns it off.

G_STEPS = 4000 — We have ~344 frauds, not ImageNet. Thousands of small steps are enough.

SYNTH_MULTIPLIER = 8 — SMOTE is told to reach the same count, so the comparison is fair.

3. data.py — split first or the project is invalid

`src/data.py`

Three jobs: (1) turn Amount into $\log_{10}(\text{Amount})$, (2) cut the table into train/val/test **before** any GAN or SMOTE, (3) fit the scaler on train only.

Amount is skewed. $\log_{10}$ squashes big purchases without going negative.

```
out = df.copy()
out["Amount"] = np.log1p(out["Amount"].astype(np.float64))
feature_names = [c for c in out.columns if c != "Class"]
X = out[feature_names].to_numpy(dtype=np.float32)
y = out["Class"].to_numpy(dtype=np.int32)
```

$\log_{10}(\text{Amount})$ — $\log(1 + \text{amount})$. Zero stays zero. 529 becomes ~6.3.

drop Class from X — Features in, label separate. GAN never sees the Class column.

float32 — What TensorFlow wants. Smaller than float64, plenty for this table.

The leakage-proof split

```
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, train_size=0.70, stratify=y, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp, train_size=0.15 / 0.30, stratify=y_tmp, random_state=42
)
```

stratify=y — Keep ~0.17% fraud in every split. Without this, test might get 20 frauds or 5.

two splits — 70% train, then the remaining 30% is cut in half → 15% val, 15% test.

random_state=42 — Same cut every run. Reproducible numbers for the README.

Scaler on train only

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # learn mean/std HERE
X_val_s = scaler.transform(X_val)          # apply, do not refit
X_test_s = scaler.transform(X_test)
```

fit_transform(train) — Mean and std come from train. That is the only 'fit'.

`transform(val/test)` — Push val/test through the same numbers. Fitting on all rows would leak.

```
def fraud_only(X, y):
    return X[y == 1]    # boolean mask: keep rows whose label is fraud
```

The GAN is trained on this slice of **X_train** only. Val frauds and test frauds never enter `src/gan.py`.

4. gan.py — the two networks

`src/gan.py` · **build_generator** / **build_critic**

Both are ordinary MLPs (Dense → LeakyReLU → Dense). This is a table of numbers, not a picture, so there are no convolutions.

Generator: noise → fake transaction

```
def build_generator(n_features, noise_dim=32):
    model = keras.Sequential(name="generator")
    model.add(keras.Input(shape=(noise_dim,)))    # z, 32 random numbers
    model.add(layers.Dense(128))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(256))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(n_features))          # 30 numbers, no sigmoid
    return model
```

Input shape (32,) — $z \sim \text{Normal}(0,1)$. Different z should make different fakes.

Dense(128) then Dense(256) — Widen, then map to the 30-D transaction.

LeakyReLU(0.2) — Lets a little negative signal through. Plain ReLU can kill units.

last layer linear — Scaled features are roughly $N(0,1)$, not 0–1. A sigmoid would squash them wrong.

n_features = 30 — Time + V1..V28 + log Amount. Same width as a real row.

Critic: transaction → score

```
def build_critic(n_features):
    model = keras.Sequential(name="critic")
    model.add(keras.Input(shape=(n_features,)))    # a real or fake row
    model.add(layers.Dense(256))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(128))
    model.add(layers.LeakyReLU(0.2))
    model.add(layers.Dense(1))                    # ONE number, not a probability
    return model
```

Dense(1), no sigmoid — This is a critic, not a discriminator. Score can be -4 or +12.

no BatchNorm — WGAN-GP needs a gradient of C with respect to the input. Batch-norm messes that up.

wider first layer — The critic is usually at least as strong as G. We give it 256 units first.

If you see `sigmoid` or `BCEWithLogitsLoss`

You accidentally wrote a vanilla GAN. Our critic has no sigmoid. Our losses are the two means below, plus GP. That is the whole difference in code.

5. gan.py — gradient penalty, line by line

`src/gan.py` · **gradient_penalty()**

This is the 'GP' in WGAN-GP. English: pick a point on the line between a real fraud and a fake, ask how steep the critic is there, and punish the critic if that slope is not close to 1.

```
def gradient_penalty(critic, real, fake):
    batch = tf.shape(real)[0]
    eps = tf.random.uniform([batch, 1], 0.0, 1.0)      # one random mix weight per row
    mixed = eps * real + (1.0 - eps) * fake            # x-hat, on the line

    with tf.GradientTape() as tape:
        tape.watch(mixed)                             # mixed is data, not a weight
        score = critic(mixed, training=True)

    grads = tape.gradient(score, mixed)                # dC/dx_hat, shape (batch, 30)
    grads = tf.reshape(grads, [batch, -1])
    norm = tf.sqrt(tf.reduce_sum(tf.square(grads), axis=1) + 1e-12)
    return tf.reduce_mean(tf.square(norm - 1.0))        # (||grad|| - 1)^2
```

eps in [0,1] — 0 means 'all fake', 1 means 'all real', 0.4 is 40% real + 60% fake.

mixed = eps*real + (1-eps)*fake — The point x-hat on that line. One per row in the batch.

tape.watch(mixed) — GradientTape tracks weights by default, not inputs. We must ask.

tape.gradient(score, mixed) — How fast the score changes if we nudge the transaction.

norm of that gradient — Steepness. We want it near 1 (1-Lipschitz).

(norm - 1)^2 — Too flat (0.2) or too steep (7.4) both get taxed. Mean over the batch.

+ 1e-12 — Stops sqrt(0) numerical junk if a gradient is exactly zero.

This number is **added** to the critic loss, multiplied by $\lambda=10$. The generator never sees it. GP only trains the critic to be a smooth judge.

6. gan.py — one critic step

src/gan.py · **WGANGP.critic_step**

@tf.function compiles this into a fast graph. Same math as a normal Python function, just quicker after the first call. GradientTape records ops so we can take derivatives.

```
@tf.function
def critic_step(self, real):
    batch = tf.shape(real)[0]
    noise = tf.random.normal((batch, self.noise_dim))
    with tf.GradientTape() as tape:
        fake = self.generator(noise, training=True)
        real_score = self.critic(real, training=True)
        fake_score = self.critic(fake, training=True)
        gp = gradient_penalty(self.critic, real, fake)
        loss = (tf.reduce_mean(fake_score)
                - tf.reduce_mean(real_score)
                + self.gp_lambda * gp)
    grads = tape.gradient(loss, self.critic.trainable_variables)
    self.c_opt.apply_gradients(zip(grads, self.critic.trainable_variables))
    return {"c_loss": loss, "gp": gp,
            "c_real": tf.reduce_mean(real_score),
            "c_fake": tf.reduce_mean(fake_score)}
```

noise = tf.random.normal(...) — Fresh z every step. G turns it into a batch of fakes.

fake = self.generator(...) — We do NOT update G here. Grads are taken only w.r.t. critic weights.

mean(fake) - mean(real) — Push fake scores down, real scores up. That is the Wasserstein critic.

+ λ * gp — Smoothness tax. λ is 10.

tape.gradient(..., critic.trainable_variables) — Only the critic's Dense kernels/biases move.

apply_gradients — Adam step on the critic. G is frozen this call.

Reading the printed numbers

After a few hundred steps you want $C(\text{real}) > C(\text{fake})$. Then critic_loss is negative (because it is fake_mean - real_mean). That is healthy, not a bug. gp should settle around 0.1–0.3, not explode to 50.

7. gan.py — one generator step (the minus sign)

src/gan.py · **WGANGP.generator_step**

This is the line you asked about. The critic outputs a score. High = 'looks real'. G wants that number **up**. TensorFlow only minimises, so we minimise the negative.

```
@tf.function
def generator_step(self, batch):
    noise = tf.random.normal((batch, self.noise_dim))
    with tf.GradientTape() as tape:
        fake = self.generator(noise, training=True)
        score = self.critic(fake, training=True)
        loss = -tf.reduce_mean(score) # ← THIS is gen_loss = -mean(C(fake))
    grads = tape.gradient(loss, self.generator.trainable_variables)
    self.g_opt.apply_gradients(zip(grads, self.generator.trainable_variables))
    return loss
```

score = critic(fake) — How real do my current forgeries look, according to C?

loss = -mean(score) — Want score high → want loss low. Minus sign is the whole trick.

grads w.r.t. generator only — Critic weights are not in this list, so C does not move this step.

no real data in this function — G cannot change C(real). It can only change the fakes.

Tiny picture: if C(fake)=3, loss=-3. G changes the fake, C now says 5, loss=-5 (better). If C says 1, loss=-1 (worse). No labels, no log, no 0/1.

```
def generate(self, n):
    noise = tf.random.normal((n, self.noise_dim))
    return self.generator(noise, training=False) # inference mode, no dropout math
```

After training we throw noise at G and save the rows. That is the fake-fraud factory. This method is also what `src/generate.py` uses via `load_model`.

8. train_gan.py — the 5:1 loop

`src/train_gan.py`

This file is the gym schedule. `gan.py` is the exercises. The GAN is handed **only** train-set frauds.

```
data = load_processed()
fraud = fraud_only(data["X_train"], data["y_train"]) # 344 rows, not 492
wgan = WGANGP(n_features=fraud.shape[1])

ds = (tf.data.Dataset.from_tensor_slices(fraud)
      .shuffle(len(fraud))
      .repeat() # we have 344 rows and 4000 steps
      .batch(64, drop_remainder=True)) # every batch exactly 64
batches = iter(ds)

for step in range(1, 4001):
    for _ in range(5): # N_CRITIC
        real = next(batches)
        c_metrics = wgan.critic_step(real)
    g_loss = wgan.generator_step(64) # once
```

fraud_only(X_train, y_train) — Val/test frauds are not here. That is the leakage rule in one line.

.repeat() — 344 rows < 4000×64. We cycle the same frauds. That is OK; there are no more.

drop_remainder=True — Last incomplete batch is dropped so the compiled graph always sees 64.

for _ in range(5): critic_step — Five judge updates, then one forger update.

generator_step once — If we updated them 1:1 the critic would lag and G would get junk gradients.

The printout every 100 steps is how you watch the fight: `c(real)` should stay above `c(fake)`. Models are saved as `generator.keras` / `critic.keras`.

9. generate.py — print the extra frauds

`src/generate.py`

```
n = 8 * len(real_fraud)                # SYNTH_MULTIPLIER
generator = tf.keras.models.load_model("artifacts/generator.keras")
noise = tf.random.normal((n, 32))
fake = generator(noise, training=False).numpy()
np.save("artifacts/synthetic_fraud.npy", fake)
```

`load_model(...)` — Reload G from disk. Critic is not needed any more.

`training=False` — Pure forward pass.

`compare column means` — Sanity check: fake V1 mean should be close to real-fraud V1 mean.

These fakes live in **scaled** space, the same space XGBoost trains in. We never inverse-transform them for the detector.

10. train_detector.py — three models, one test set

`src/train_detector.py`

The science is the comparison, not the GAN by itself. Same XGBoost hyperparameters. Only the **training rows** change. Thresholds are picked on val, then test is scored once.

Shared fit-and-score helper

```
val_p = model.predict_proba(X_val)[: , 1]    # P(fraud) on val
test_p = model.predict_proba(X_test)[: , 1]
thr, val_f1 = best_f1_threshold(y_val, val_p)  # pick cutoff HERE
report = {"test": full_report(y_test, test_p, thr)} # apply once HERE
```

`[: , 1]` — Class 1 = fraud. `predict_proba` returns `[P(legit), P(fraud)]`.

`best_f1_threshold(y_val, ...)` — Search cutoffs on val only. Test is not a tuning knob.

`full_report(..., thr)` — Precision/recall/F1 at that frozen cutoff, plus AUPRC on the raw scores.

Model 1 — class weights, no extra rows

```
w = n_neg / n_pos                # ~199k / 344 ≈ 579
model = XGBClassifier(scale_pos_weight=w, ...)
model.fit(X_tr, y_tr)             # original train, nothing synthetic
```

`scale_pos_weight` tells the tree: a missed fraud costs $\sim 579\times$ a missed legit. This is the 'do nothing fancy' baseline.

Model 2 — SMOTE to the same minority count

```
target_pos = n_pos + 8 * n_pos    # 344 + 2752 = 3096
smote = SMOTE(sampling_strategy={1: target_pos}, k_neighbors=5)
X_sm, y_sm = smote.fit_resample(X_tr, y_tr)
model.fit(X_sm, y_sm)
```

`sampling_strategy={1: 3096}` — After SMOTE, class 1 must have 3096 rows. SMOTE interpolates to get there.

`k_neighbors=5` — Each new point sits on a line toward one of 5 nearest real frauds.

`same target as GAN` — If SMOTE made 50k fakes and GAN made 2k, the comparison would be a lie.

Model 3 — real train + GAN fakes

```
fake = np.load("artifacts/synthetic_fraud.npy")[:n_needed]
X_gan = np.vstack([X_tr, fake])      # stack rows
y_gan = np.concatenate([y_tr, np.ones(len(fake))]) # label every fake as fraud
model.fit(X_gan, y_gan)
```

`np.ones(len(fake))` — We told G to copy the fraud blob, so we label fakes as 1. They never go into test.

`vstack with full X_tr` — Keep every legit row. We oversample minority; we do not throw away majority.

`no scale_pos_weight here` — Oversampling is the balancing. Adding both would double-count.

11. evaluate.py — AUPRC and the cutoff

`src/evaluate.py`

```
def threshold_free(y_true, proba):
    return {
        "auprc": average_precision_score(y_true, proba), # primary metric
        "roc_auc": roc_auc_score(y_true, proba),
    }

def best_f1_threshold(y_true, proba):
    prec, rec, thr = precision_recall_curve(y_true, proba)
    f1 = 2 * prec[:-1] * rec[:-1] / (prec[:-1] + rec[:-1] + 1e-12)
    i = int(f1.argmax())
    return float(thr[i]), float(f1[i])
```

average_precision_score — Area under precision-recall. Does not need a cutoff. Put this on the README.

precision_recall_curve — Every possible cutoff, with its precision and recall.

prec[:-1] — sklearn returns one extra (1, 0) point with no matching threshold. Drop it.

argmax F1 on val — One operating point for the confusion matrix. Not used to brag about AUPRC.

Why accuracy never appears

A model that predicts 'not fraud' for every row is correct on 99.83% of this dataset and catches zero thieves. That is why `evaluate.py` does not import `accuracy_score`.

12. infer.py — the GAN is not in production

`src/infer.py`

This is the sentence that carries an interview: live scoring loads the scaler and XGBoost. It does not load `generator.keras`.

```
model = XGBClassifier()
model.load_model("artifacts/xgb_gan.json") # detector, not the GAN
Xs = scaler.transform(X_raw) # same scaler as training
proba = model.predict_proba(Xs)[:, 1]
flag = proba >= threshold # val-chosen, frozen
```

load xgb_gan.json — The tree trained on real+fake. G already did its job offline.

scaler.transform — New rows must take the same $\log_{10}(\text{Amount}) + \text{StandardScaler}$ path.

proba >= threshold — 0.746 on our run. Below that = legit, above = flag.

13. GradientTape in one paragraph

You will see with `tf.GradientTape()` as `tape: twice`. Everything computed inside that block is recorded. `tape.gradient(loss, variables)` asks: if I nudge each variable, how does loss change? Then Adam takes a small step opposite that direction (because we minimise).

Two tapes, two sets of variables:

- Critic step: loss depends on C's weights → update C.
- Generator step: loss depends on G's weights (through $\text{fake} = G(z)$) → update G.

The inner tape in `gradient_penalty` is different: it differentiates C's **score** with respect to the **input row**, not the weights. That slope is the Lipschitz check.

14. Trace one fake row through the system

Stage	What happens to the row
$z \sim N(0,1)$	32 random numbers. No meaning yet.
$G(z)$	Becomes a 30-D vector in scaled feature space.
$C(G(z))$	During training only: a score. G tries to raise it.
<code>np.save</code>	After training, 2752 such rows hit <code>synthetic_fraud.npy</code> .
<code>vstack</code>	They are glued under the real train matrix, labelled 1.

Stage	What happens to the row
XGB.fit	The tree sees them as extra fraud examples.
test / infer	They are gone. A live transaction never meets G.

You understand the code when you can say this without the PDF

data.py splits first and scales on train. gan.py is two MLPs: $G(z) \rightarrow \text{row}$, $C(\text{row}) \rightarrow \text{score}$. Critic loss is $\text{mean}(\text{fake}) - \text{mean}(\text{real}) + 10 \cdot \text{GP}$. Generator loss is $-\text{mean}(C(\text{fake}))$. train_gan.py does that 5:1, on train frauds only, 4000 times. Fakes are stacked onto train, XGBoost is the detector, the cutoff is chosen on val, test is scored once, infer.py does not load G.

Pair this with the crash-guide PDF for the words (Wasserstein, Lipschitz, AUPRC). This file is only the code those words turned into.